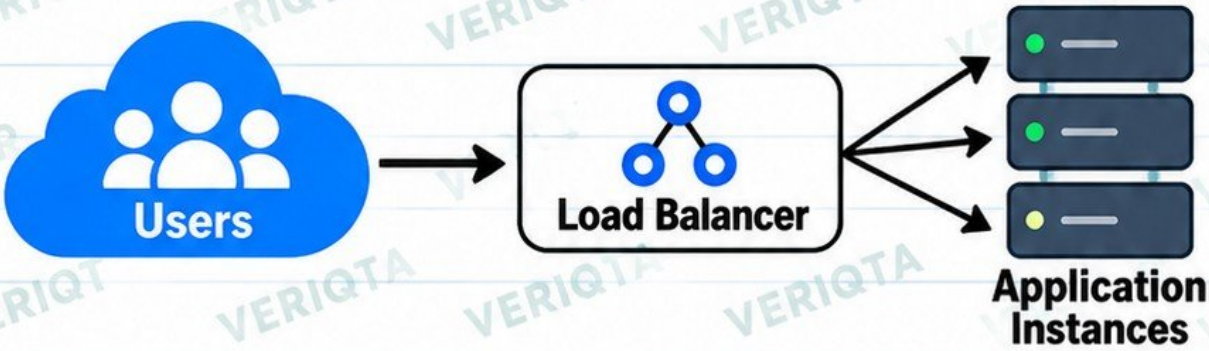
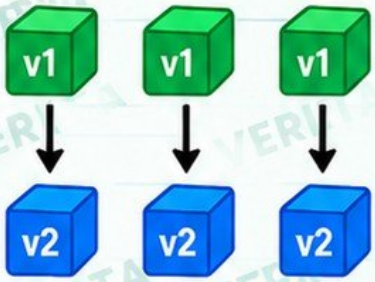
VERIQTA

Zero-Downtime Deployments Explained:

Rolling Updates, Blue-Green, and Canary Deployments

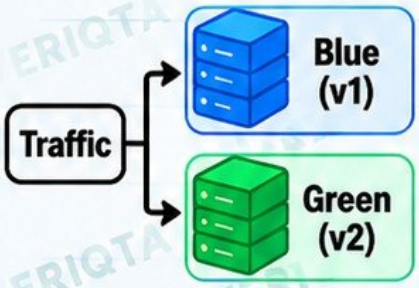


Rolling Updates



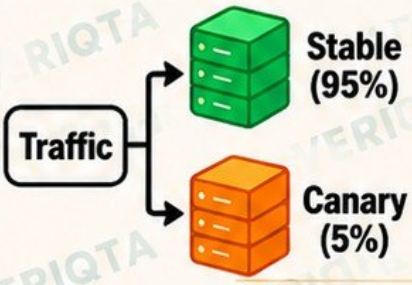
Replace instances gradually while keeping the service available.

Blue-Green



Prepare the new version separately, then switch traffic to it.

Canary



Send a small portion of traffic to the new version and increase gradually.

@veriqta | @veriqta | @veriqta | @veriqta | @veriqta | @veriqta

Page title

What Does Zero-Downtime Deployment Really Mean?

VERIQTA

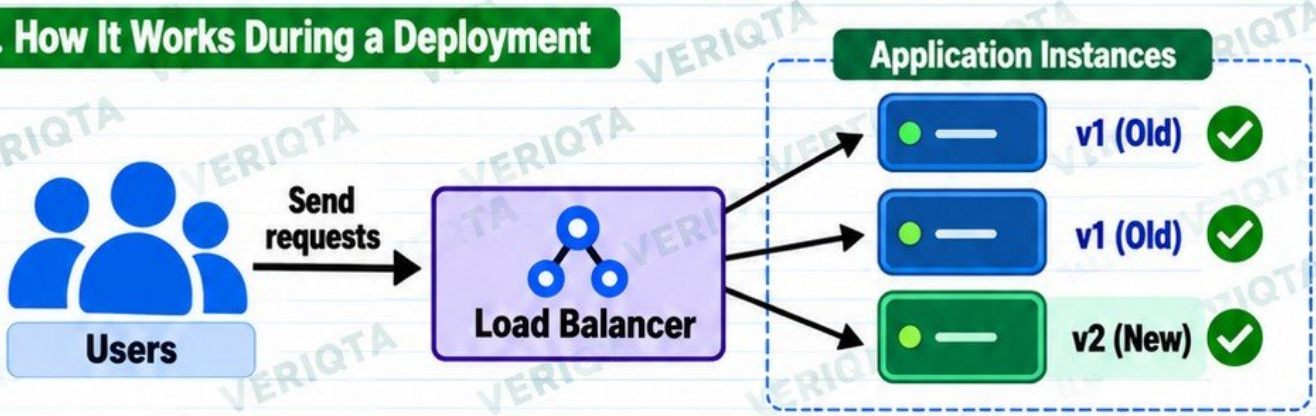
1. What Is Zero-Downtime Deployment?

- A deployment **without planned service interruption.**
- Users can continue to access the application while a new version is released.
- It keeps the service available even while infrastructure or application instances are being updated.

2. Key Points

- **Availability** means the service is reachable and working for users.
- **Successful deployment completion** means the new version is fully deployed.
- A deployment can complete successfully but the application can still have **errors** afterwards.

3. How It Works During a Deployment



While the new version is being deployed, the load balancer sends traffic only to healthy instances. Users keep accessing the application.

4. Zero Downtime Does Not Mean Zero Errors

- During a deployment, some errors can still occur such as:
  - a few slow responses
  - temporary errors while new instances start
  - errors in the new version
  - compatibility issues between old and new versions
- The goal is to avoid **planned service interruption**, not to guarantee that every single request will be error-free.

5. Important Distinction

Service Availability

- Users can reach the application
- Requests are served
- No planned downtime

≠

Deployment Completion

- New version is fully deployed
- All instances updated
- Deployment process has finished

Junior Engineer Insight: A deployment is not successful simply because the new version is running. It is successful when the new version is running and users continue to access a healthy and stable service.

Why Deployments Cause Outages

VERIQTA

1. What Happens in an Unsafe Deployment?

1. Engineers stop the old application before starting the new one.
2. All running instances are terminated or removed from the load balancer.
3. There are no healthy instances to serve user requests.
4. Users experience errors until the new version becomes available.

2. Common Reasons for Deployment Failures

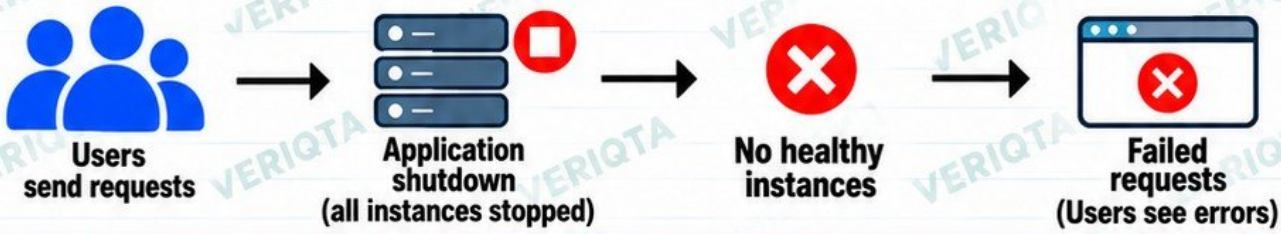
Startup Delays
New instances take time to start. During this time, there may be no healthy instances to serve requests.

Failed Health Checks
New instances may fail readiness checks due to misconfiguration or missing dependencies, so they do not receive traffic.

Broken Dependencies
The new version may depend on databases, external services, or configuration that is not ready or compatible.

Dropped Connections
Active user connections can be terminated when instances are stopped, causing failed requests.

3. What an Outage Looks Like



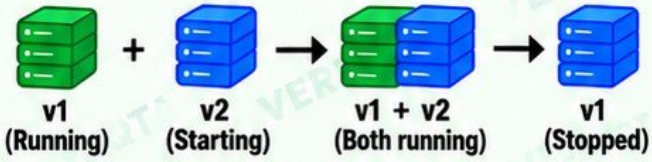
4. Unsafe Stop-and-Replace vs Overlapping Deployment

Unsafe Stop-and-Replace Deployment



- Old instances are stopped before new ones are ready.
- There is a period with no healthy instances.
- Users receive errors during this time.

Overlapping Deployment (Safe Approach)



- New instances are started and become healthy first.
- Both old and new versions run together temporarily.
- Traffic continues to flow to healthy instances.
- Old instances are removed only after the new version is confirmed healthy.

5. Common Failure

Every application instance becomes unavailable simultaneously.

This can happen when all instances are stopped, fail health checks, or are removed from the load balancer before new instances are ready.

6. Junior Engineer Insight

A deployment is not successful simply because the new version is running. It is successful when users can access the application without service interruption and it remains healthy after the deployment.

The Building Blocks of Safe Deployments

VERIQTA

1. Key Building Blocks

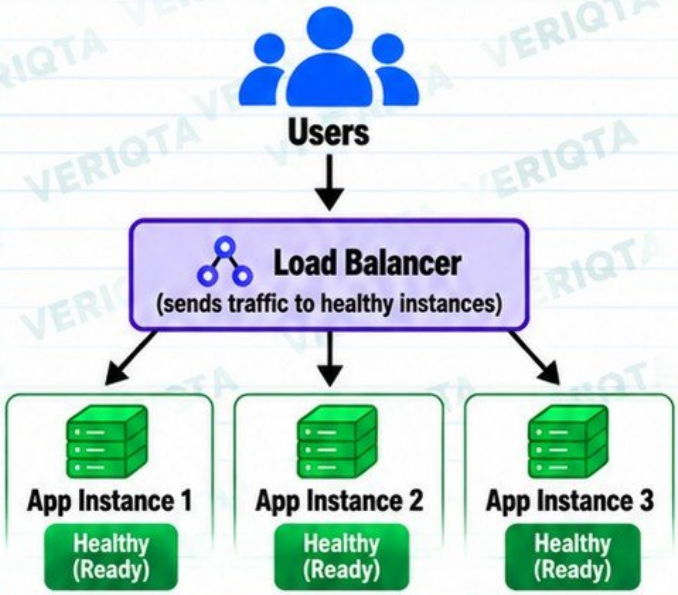
Replicas
Multiple instances of your application running at the same time.

Load Balancer
Distributes incoming traffic to healthy application instances.

Readiness Checks
Verify that an instance is ready to receive production traffic.

Traffic Routing
Sends user requests only to healthy and ready instances.

2. How It Works Together



3. Why Multiple Instances Alone Is Not Enough

- Instances may be running but **not healthy**.
- Instances may **fail at the same time** due to a bad deployment.
- A load balancer may still send traffic to **unhealthy instances** if readiness checks are not configured.
- **Dependencies** such as databases or external services can still be **unavailable**.
- Without proper traffic routing and health checks, users can still **experience errors** even with multiple instances.

4. Capacity Planning

- Run enough replicas to handle expected traffic.
- Plan extra capacity during deployments (because some instances may be starting or stopping).
- Consider peak traffic, not just average traffic.
- Make sure your infrastructure can support both old and new versions during the deployment.

5. Graceful Shutdown

- Allow the application to finish ongoing requests before stopping.
- Remove the instance from the load balancer first.
- Give the instance time to complete active connections.
- Prevent dropped connections and user errors.

6. Production Awareness

An application must be ready to serve requests before receiving production traffic.

- Use readiness checks to control when an instance receives traffic.
- Verify the application is fully initialized.
- Monitor new instances after deployment.
- Remove unhealthy instances from traffic immediately.

The Deployment Lifecycle: Before, During, and After

1. Before Deployment



- Test the new version in a staging environment.
- Validate compatibility (with databases, APIs, configurations, etc).
- Check capacity and ensure you have enough resources.
- Prepare a rollback plan in case something goes wrong.

2. During Deployment



- Start new application instances (Pods/containers).
- Check readiness before sending traffic.
- Control traffic using a deployment strategy (rolling, blue-green, or canary).
- Monitor the deployment progress and application health.

3. After Deployment



- Verify the application behaves as expected.
- Monitor errors, latency, and key metrics.
- Confirm the deployment is fully completed.
- Keep observing for a period of time to catch any issues that appear later.

4. The Deployment Lifecycle Flow

Prepare



Test, validate, check capacity, prepare rollback

→

Deploy



Start new instances and check readiness

→

Route



Send traffic to healthy and ready instances

→

Observe



Monitor application behavior, errors and performance

→

Verify



Confirm successful deployment and stable service

5. Deployment Success vs Release Health

Deployment Success

- The new version is fully deployed.
- All instances are updated (according to the strategy).
- Deployment process has finished without errors.
- Kubernetes or the deployment tool shows the rollout is complete.

Release Health

- Users can successfully use the application.
- Error rates are low and within normal levels.
- Latency and performance are normal.
- All key features work as expected.
- No new issues after the release.

6. Practice: When Should the Deployment Stop?

You are deploying a new version. After a few instances are created, the readiness checks fail and the instances are not receiving traffic.

What should you do?

- A Continue the deployment anyway.
- B Stop or pause the deployment.
- C Remove the old version immediately.
- D Ignore the failed readiness checks.

Correct Answer: B

You should stop or pause the deployment and investigate the issue before proceeding.

7. Key Takeaways



- A deployment is not complete simply because the new version is running.
- The application must be ready to serve requests before receiving production traffic.
- Always test, monitor, and have a rollback plan.
- Zero downtime is an operational goal, but you must verify release health after deployment.

Rolling Updates: Replace Instances Gradually

VERIQTA

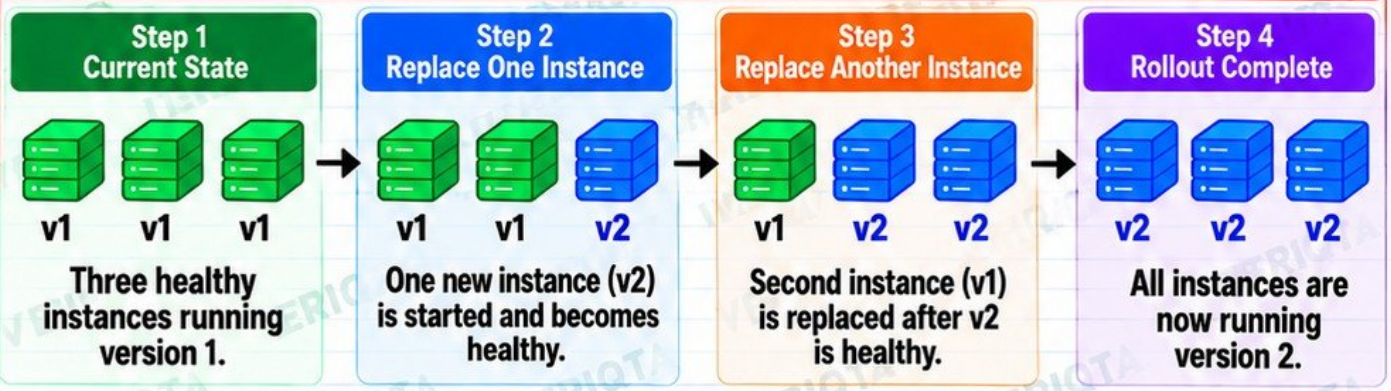
1. What Is a Rolling Update?

- A rolling update replaces old application instances with new ones gradually, one or a few at a time.
- The old and new versions run together during the deployment.
- Users continue to access the application while instances are updated.
- The deployment is completed when all old instances are replaced with new ones.

2. Why It Works

- A load balancer sends traffic only to healthy instances.
- New instances are started and checked for readiness before they receive traffic.
- Old instances are removed gradually after new ones are healthy.
- This keeps enough healthy instances available to serve user requests.

3. How a Rolling Update Replaces Instances (Example with 3 Replicas)



4. Why Capacity and Compatibility Matter

- You need enough capacity to run old and new instances at the same time.
- If capacity is too low, you may not have enough healthy instances, which can cause outages.
- The new version must be compatible with the old version, especially when they share databases, caches, or other services.
- Proper readiness checks ensure new instances are actually ready before receiving traffic.

5. Key Benefits

- No planned service interruption.
- Users continue to access the application.
- Risk is reduced because changes are rolled out gradually.
- If a problem is detected, you can stop or rollback the deployment.

6. Beginner Mistake

Assuming every new instance becomes healthy immediately.

New instances can take time to start, may fail readiness checks, or may crash due to configuration or dependency issues. Always verify that an instance is healthy before allowing it to receive traffic.

7. Junior Engineer Insight

A rolling update is not just about replacing instances. It is about maintaining enough healthy capacity, verifying readiness, and ensuring compatibility so users continue to receive a stable service throughout the deployment.

Kubernetes Rolling Updates: How Deployments Work

VERIQTA

1. Key Kubernetes Resources

Deployment
- Manages the desired state of your application.
- Handles rolling updates automatically.

ReplicaSet
- Maintains the desired number of Pods.
- Created and managed by the Deployment.

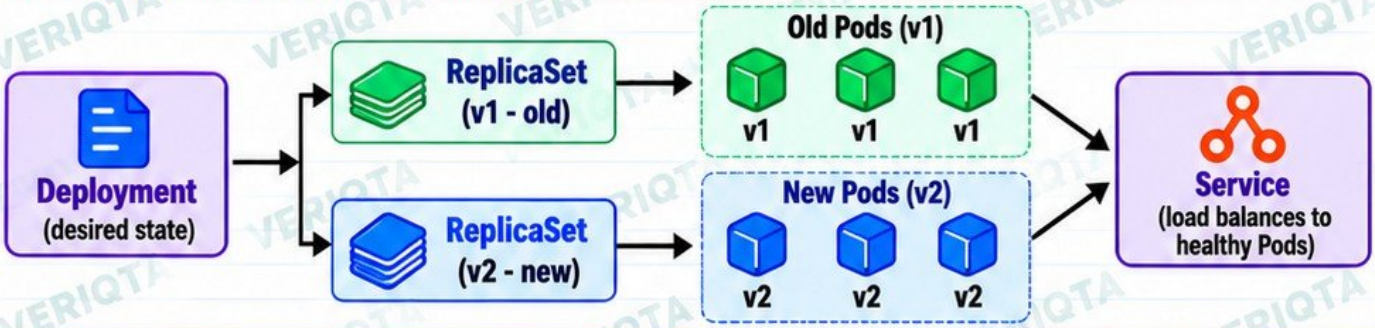
Pod
- A running instance of your application.
- Can be created, replaced, or terminated.

Service
- Provides a stable endpoint and load balances traffic to healthy Pods.

2. How Rolling Updates Work

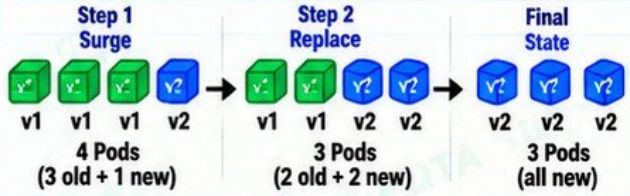
1. You update the Deployment with a **new image**.
2. Kubernetes creates a **new ReplicaSet** for the new version.
3. **New Pods** are created from the new ReplicaSet.
4. Kubernetes gradually **terminates old Pods**.
5. Both old and new Pods run at the **same time** during the rollout.
6. The Service continues to send traffic to **healthy Pods**.
7. The rollout is complete when all old Pods are replaced.

3. Rolling Update Flow in Kubernetes



4. maxSurge and maxUnavailable (3 Replicas Example)

maxSurge = 1, maxUnavailable = 0
- Start 1 extra new Pod (total 4 Pods).
- Wait until the new Pod is healthy.
- Terminate 1 old Pod (back to 3 Pods).
- Repeat until all Pods are updated.



maxSurge = 0, maxUnavailable = 1
- Terminate 1 old Pod first (2 Pods remain).
- Start 1 new Pod (back to 3 Pods).
- Wait until the new Pod is healthy.
- Repeat until all Pods are updated.



5. Important Notes
- **maxSurge** controls how many extra Pods can be created.
- **maxUnavailable** controls how many Pods can be unavailable during the rollout.
- A higher surge can make rollouts faster but requires more capacity.
- A higher unavailable value can reduce resource usage but increases risk.

6. DevOps Connection

Kubernetes coordinates the rollout by creating and deleting Pods, but **application readiness** determines whether new Pods should receive traffic. If the application is not ready, users can still experience errors even though the rollout is progressing.

Your First Kubernetes Rolling Update

VERIQTA

1. Starting Point: A Healthy Deployment

- We already have a running application with 3 replicas.
- The application is accessible and healthy.
- We will update it to a new version using a rolling update.

2. Inspect the Current Deployment

```
kubectl get deployment myapp
NAME    READY   UP-TO-DATE   AVAILABLE   AGE
myapp   3/3     3            3           10m
```

- Check the current image used by the Deployment.

```
kubectl get deployment myapp -o wide
NAME    READY   UP-TO-DATE   AVAILABLE   IMAGE          AGE
myapp   3/3     3            3           nginx:1.0      10m
```

3. Update the Image (Rolling Update)

- Use kubectl set image to update to a new version.

```
kubectl set image deployment/myapp \
  myapp-container=nginx:2.0
```

- deployment/myapp → name of the Deployment
- myapp-container → name of the container in the pod
- nginx:2.0 → new image version

4. Observe the Rollout Progress

- Watch the rollout status to see the update in real time.

```
kubectl rollout status deployment/myapp
Waiting for deployment "myapp" rollout to finish...
deployment "myapp" successfully rolled out
```

5. Verify the New Image

- Check the deployment again.

```
kubectl get deployment myapp
NAME    READY   UP-TO-DATE   AVAILABLE   AGE
myapp   3/3     3            3           15m
```

- Confirm the new image is now in use.

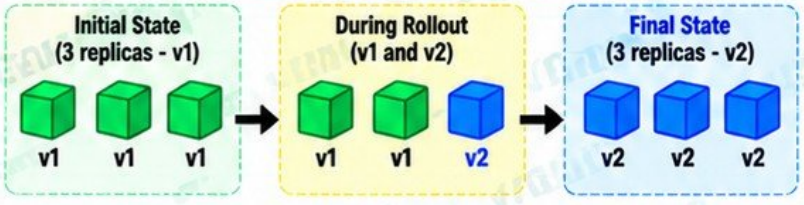
```
kubectl get deployment myapp -o wide
NAME    READY   UP-TO-DATE   AVAILABLE   IMAGE          AGE
myapp   3/3     3            3           nginx:2.0      15m
```

6. See Old and New Pods During Rollout

- While the rollout is in progress, you can see old and new Pods running together.

```
kubectl get pods -l app=myapp -w
NAME                     READY   STATUS    AGE   VERSION
myapp-7d6c9f8b7-abc12    1/1     Running   2m    1.0
myapp-7d6c9f8b7-def34    1/1     Running   2m    1.0
myapp-7d6c9f8b7-ghi56    1/1     Running   2m    1.0
myapp-5f8d9c6c4-jk178    1/1     Running   20s   2.0
myapp-5f8d9c6c4-mno90    1/1     Running   20s   2.0
myapp-5f8d9c6c4-pqr12    1/1     Running   20s   2.0
```

7. Diagram: Rolling Update in Action



8. Practice Lab

1. Start with a 3-replica Deployment (e.g. nginx).
2. Check the current image and Pods.
3. Update the image to a new version using kubectl set image.
4. Watch the rollout with kubectl rollout status.
5. Observe old and new Pods running together.
6. Verify the new image and confirm all Pods are updated.

9. Key Takeaways

- Kubernetes replaces old instances in batches, not all at once.
- Old and new versions run together during the rollout.
- The new Pods must be ready before they receive traffic.
- Always verify the rollout and check the application is healthy.

10. Beginner Mistake

Assuming every new instance becomes healthy immediately.

New Pods can take time to start. If they are not ready, they will not receive traffic and the rollout may pause or fail based on readiness checks.

When a Rolling Update Goes Wrong

VERIQTA

1. Common Reasons Pods Fail

- Application errors (crashes on startup).
- Failed readiness checks.
- Missing or incorrect environment variables.
- Cannot connect to a database or external service.
- Image pull errors (wrong image name or private registry).
- Insufficient resources (CPU or memory).
- Configuration or compatibility issues.

2. Investigate a Pod That Is Not Ready

Get more details about the failed Pod.

```
kubectl describe pod <pod-name>
```

```
Name:         myapp-7d6c9f8b7-abc12
Namespace:    default
Status:       Pending
Reason:       CrashLoopBackOff
Message:      Back-off restarting failed container
```

3. Check Pod Logs

- View the application logs to find the error.

```
kubectl logs <pod-name>
```

```
Error: failed to connect to database
Caused by: dial tcp 10.0.0.10:5432: connect
: connection refused
```

4. Check the Rollout Status

- See the current status of the deployment.

```
kubectl rollout status deployment/myapp
```

```
Waiting for deployment "myapp" rollout to finish...
0 of 3 updated replicas are available ...
```

5. Stalled Rollouts and progressDeadlineSeconds

- A rollout can become stuck if the new Pods never become ready.
- Kubernetes waits for a certain time, defined by **progressDeadlineSeconds** (default is 600 seconds).
- If no progress is made within this time, the deployment is marked as failed.

```
kubectl get deployment myapp -o yaml | grep
progressDeadlineSeconds
```

↳ `progressDeadlineSeconds: 600`

6. Undo a Failed Rollout

- Roll back to the previous working version.

```
kubectl rollout undo deployment/myapp
```

```
deployment.apps/myapp rolled back
```

- You can also check the rollout history:

```
kubectl rollout history deployment/myapp
```

```
REVISION    CHANGE-CAUSE
1           <none>
2           kubectl set image deployment/myapp ...
```

7. Example: Rollout Stuck Due to Unready Pods



⚠ Rollout stops because the new Pods fail to become ready.
Old Pods may be terminated, leaving no healthy instances.

8. Common Failure

The rollout stops progressing because the new application never becomes ready.

9. Production Awareness

Kubernetes does not automatically undo every failed rollout. You must investigate the issue and manually take action, such as fixing the problem or using kubectl rollout undo.

10. Key Takeaways

- Use **kubectl describe pod** and **kubectl logs** to find the cause.
- Monitor the rollout with **kubectl rollout status**.
- Understand **progressDeadlineSeconds** and why a rollout fails.
- Use **kubectl rollout undo** to return to the previous version.
- Always verify that new Pods are ready before old ones are removed.

Blue-Green Deployments: Two Environments, One Live

VERIQTA

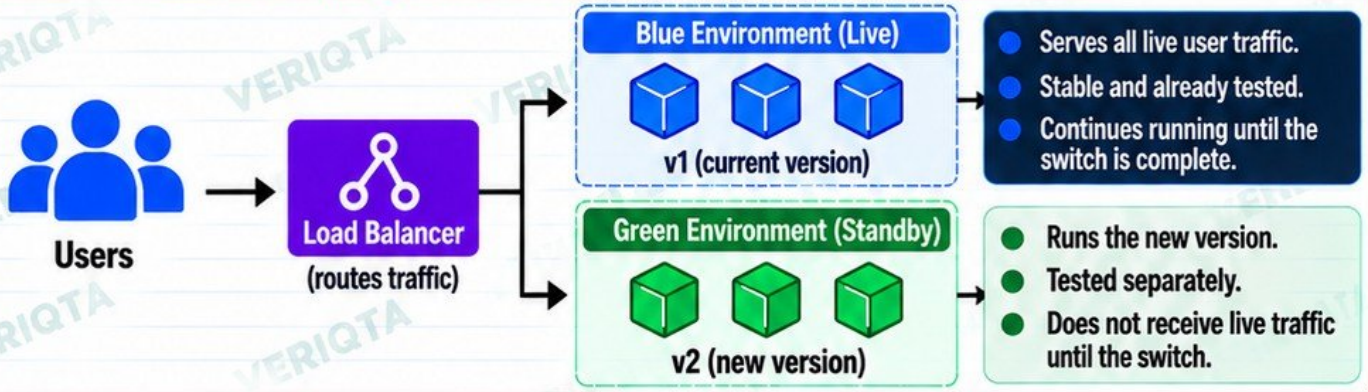
1. What Is a Blue-Green Deployment?

- Use two identical environments (Blue and Green).
- One environment (Blue) serves all live users.
- The other environment (Green) runs the new version.
- When the new version is ready, traffic is switched from Blue to Green.
- Both environments must be compatible with the same databases, caches, and external services.

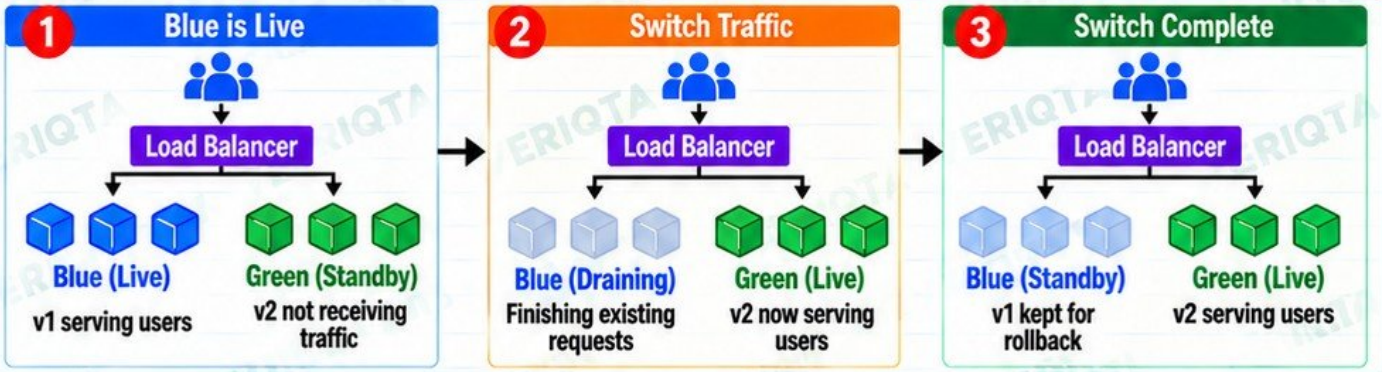
2. Key Benefits

- No service interruption during the switch.
- Full testing in the Green environment before it receives live traffic.
- Quick rollback by switching back to Blue.
- Reduced risk compared to updating a single environment.
- Easy to test the new version with real-like conditions.

3. How Blue-Green Deployments Work



4. Switching Traffic from Blue to Green



5. Important Considerations

- Both environments must be compatible with shared dependencies (databases, caches, external services).
- Database schema changes must work with both versions during the switch.
- Keep Blue running for a period after the switch in case rollback is needed.
- Infrastructure, configuration, and environment variables must match.

6. Junior Engineer Insight

Preparing the Green environment separately reduces exposure during testing, but switching traffic still carries risk. You must verify the application is healthy after the switch and be ready to roll back if needed.

7. Common Risks

- Undetected issues in the Green environment.
- Incompatible database changes.
- Configuration differences.
- Traffic switch failures.
- Assuming the new version works without proper testing.

How Blue-Green Traffic Switching Works

VERIQTA

1. How Traffic Reaches Your Application

- Users send requests to a domain or IP address.
- A load balancer, Kubernetes Service, or traffic management system routes the requests.
- The router forwards traffic to the healthy backend instances.
- In a blue-green setup, traffic is initially sent to the Blue environment.
- When you switch, the router starts sending traffic to the Green environment.

2. Ways to Switch Traffic

1 Kubernetes Service (label selector)

- Change the Service selector to point to Green Pods.
- New requests are sent to the new set of Pods.

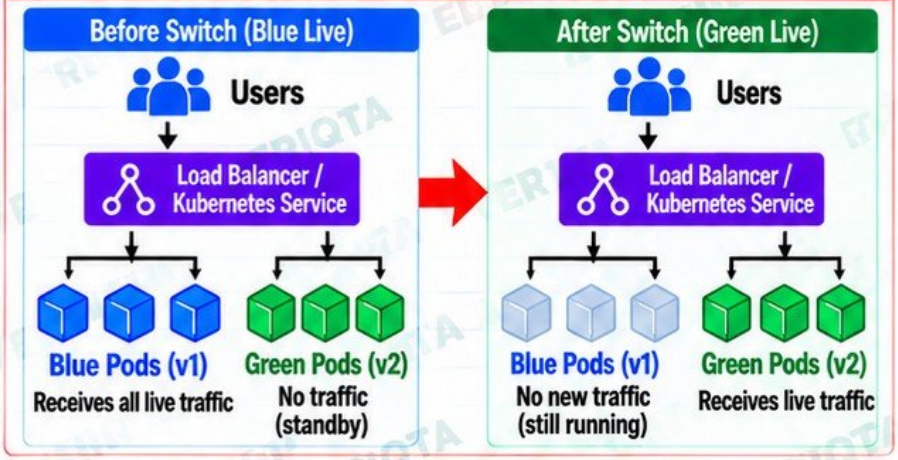
2 Load Balancer / Ingress Controller

- Update the backend targets from Blue to Green.
- Can be done through your cloud provider or ingress configuration.

3 DNS-Based Cutover

- Update the DNS record to point to the Green environment.
- New requests go to Green after DNS propagation.
- There can be a delay while DNS changes propagate across the internet.

3. Before and After the Switch (Diagram)



4. What Happens During a Switch

- New requests are routed to Green.
- Existing connections to Blue may continue until they complete.
- Load balancers and ingress controllers may use connection draining to let existing requests finish.
- With DNS-based cutover, users may still reach Blue for a short time due to propagation delays and DNS caching.
- Both environments must remain available during the switch to avoid service interruption.

5. Kubernetes Service Selector Example

- The Service routes traffic to Pods that match the selector labels.

Before (points to Blue)

```
apiVersion: v1
kind: Service
metadata:
  name: myapp
spec:
  selector:
    version: blue
```

After (points to Green)

```
apiVersion: v1
kind: Service
metadata:
  name: myapp
spec:
  selector:
    version: green
```

6. Application vs DNS Switching

Application Traffic Switching (Load Balancer or Kubernetes Service)

- Changes backend targets or Service selector.
- New requests go to Green immediately.
- Existing connections may continue to Blue until they finish.
- Controlled by your infrastructure and happens quickly.

DNS-Based Cutover

- Update the DNS record to point to Green.
- Depends on DNS propagation and caching.
- Some users may still reach Blue for a period of time.
- Useful for switching between completely separate environments.

7. Connection Draining

- Load balancers can stop sending new requests to Blue.
- Existing connections are allowed to complete.
- This reduces the risk of dropping in-flight requests.
- The time to drain connections depends on your load balancer settings and application behaviour.

8. Common Mistake

Assuming all existing requests immediately move to Green.

In reality, existing connections may continue to Blue, and DNS changes can take time to fully propagate.

Blue-Green Deployment with Kubernetes

VERIQTA

1. Create Two Application Deployments

- Create a Blue deployment (current version).
- Create a Green deployment (new version).
- Use version labels to identify each environment.

Blue Deployment (v1)

```
kubectl create deployment
myapp-blue --image=nginx:1.0
--replicas=3 --label=app=myapp,
version=blue
```

Green Deployment (v2)

```
kubectl create deployment
myapp-green --image=nginx:2.0
--replicas=3 --label=app=myapp,
version=green
```

2. Create a Kubernetes Service (Active Version)

- Create a single Service to route traffic.
- Set the selector to the active version (Blue).
- Only Pods with matching labels receive traffic.

Service (initially points to Blue)

```
apiVersion: v1
kind: Service
metadata:
  name: myapp
spec:
  selector:
    app: myapp
    version: blue      # routes to Blue Pods
  ports:
  - port: 80
    targetPort: 80
```

3. Verify Green is Ready Before Switching

- Check that Green Pods are running and ready.
- Make sure the application works as expected.

```
# Check Green Pods
kubectl get pods -l app=myapp,version=green

# Check readiness
kubectl get pod -l app=myapp,version=green

# Test the application (use port-forward or internal URL)
kubectl port-forward svc/myapp 8080:80
```

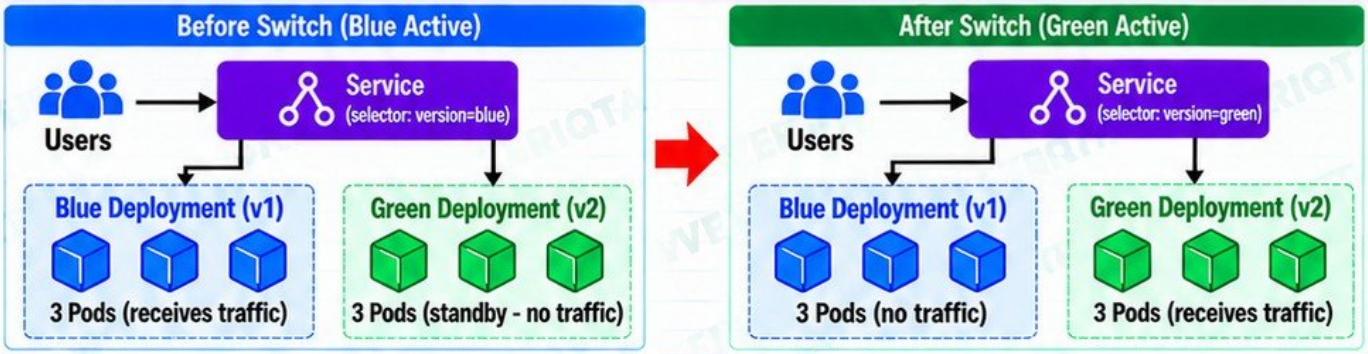
4. Observe Service Endpoints and Test Responses

- Check which Pods the Service is sending traffic to.
- Test the application and confirm the current version.

```
# View Service endpoints
kubectl get endpoints myapp

# Test the application (should return Blue version initially)
curl http://<service-ip>
# or use port-forward
curl http://localhost:8080
```

5. Diagram: Blue to Green with Kubernetes Service



6. Switch the Service Selector to Green

- Update the Service selector to point to Green.
- New requests will be sent to the Green Pods.

```
kubectl patch service myapp -p '{
  "spec": {
    "selector": {
      "app": "myapp",
      "version": "green"
    } }
}'
```

7. Practice Lab

1. Create Blue deployment with 3 replicas.
2. Create Green deployment with 3 replicas.
3. Verify Green is ready and working.
4. Check Service endpoints (should point to Blue).
5. Update the Service selector to Green.
6. Test the application and confirm the new version.
7. Keep the Blue deployment running (do not delete).

8. Security Reminder

Avoid exposing the standby environment publicly unless access is deliberately controlled.

- Use internal Services or restricted access.
- Limit access with network policies, firewalls, or authentication.
- The standby environment may contain unfinished or sensitive changes.

9. Key Takeaways

- Use two identical environments with version labels.
- A single Service can switch traffic by changing the selector.
- Always verify the new version is ready before switching.
- Keep the old environment running for quick rollback.

Blue-Green Rollback and Hidden Risks

VERIQTA

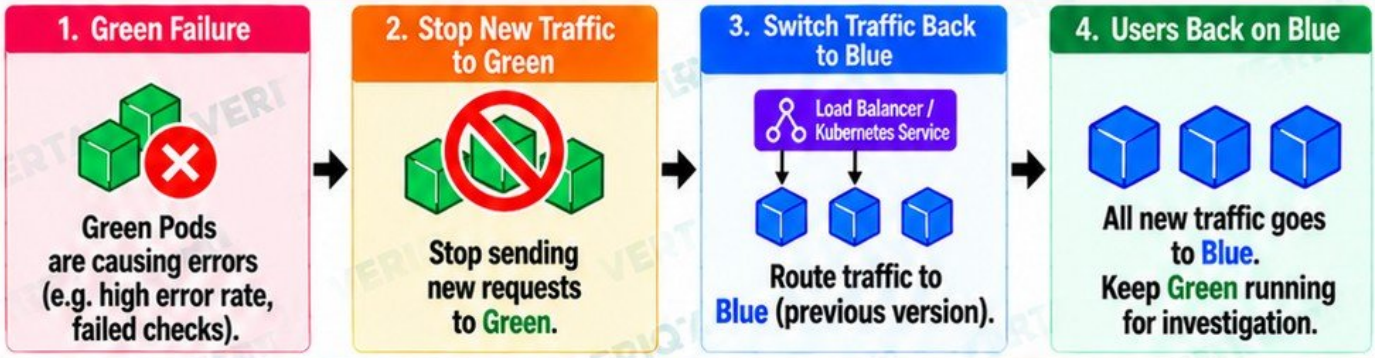
1. When to Roll Back to Blue

- Roll back when the Green environment causes errors or unexpected behavior.
- Stop sending new traffic to Green.
- Switch the Service or load balancer back to Blue.
- Keep Green running while you investigate the issue.
- Communicate the rollback to your team and monitor closely.

2. How to Roll Back Traffic

1. Identify the issue in Green (errors, high latency, failed checks).
2. Stop sending new traffic to Green by changing the Service selector or load balancer target.
3. Verify that Blue is healthy.
4. Switch traffic back to Blue.
5. Monitor the application to confirm recovery.

3. Diagram: Rollback from Green to Blue



4. Conditions That Make Rollback Unsafe

- Database schema changes that are not backward compatible.
- Data migrations that cannot be easily reversed.
- Irreversible changes to external systems (payments, emails, etc.).
- Application code that is not compatible with the old version.
- Background jobs that process data in a new format.

5. Hidden Risks in Blue-Green Deployments

- **Session storage:** User sessions may be lost or invaiid when switching between versions.
- **Background jobs:** Jobs may continue running in Green and process data in a new format.
- **Caches:** Cached data may be incompatible between versions.
- **Shared dependencies:** Both environments use the same databases, queues, and external services.
- **Partial rollout:** Some users or systems may still reach Green due to caching or DNS delays.

6. Parallel Capacity Costs

- Both Blue and Green environments run at the same time.
- This doubles (or nearly doubles) infrastructure, monitoring, and operational costs.
- You pay for compute, storage, networking, and other resources for both environments.
- Plan and budget for the extra cost before using blue-green deployments in production.

7. Production Awareness

A traffic switch cannot reverse database changes or external side effects.

- Rolling back traffic only affects where new requests go.
- It does not undo data changes, sent emails, processed payments, or other external actions.
- Always assess the impact of changes before switching traffic.

8. Key Takeaways

- You can quickly switch traffic back to Blue if Green has issues.
- Rollback is not always safe, especially with database changes.
- Consider sessions, background jobs, caches, and shared services.
- Maintaining two environments increases cost.
- Plan for risks and test rollback procedures in a non-production environment.

Canary Deployments: Release to a Small Audience First

1. What Is a Canary Deployment?

- A canary deployment releases a new application version to a small portion of production traffic.
- Most users continue to use the stable version.
- A small group of users are routed to the canary version.
- Engineers monitor the canary for errors, performance issues, and unexpected behavior.
- If the canary is healthy, traffic is gradually increased until all users are on the new version.

2. Stable Version vs Canary Version

Stable Version (v1)



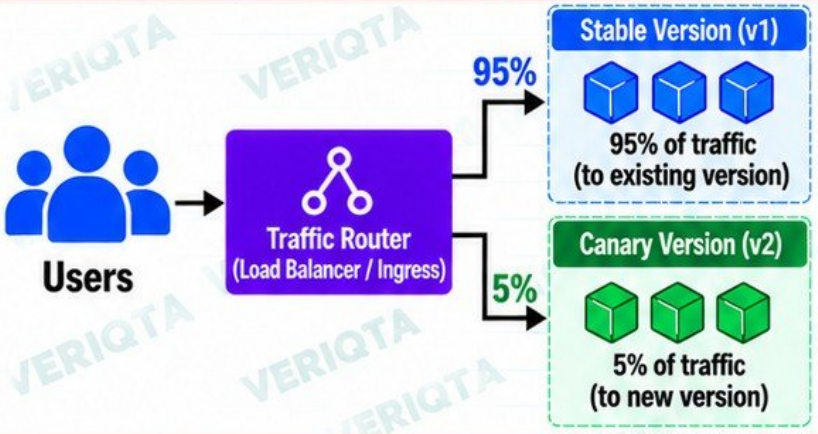
- Handles most of the production traffic.
- Proven and stable.
- Known performance and behavior.
- Continues to serve users during the rollout.

Canary Version (v2)

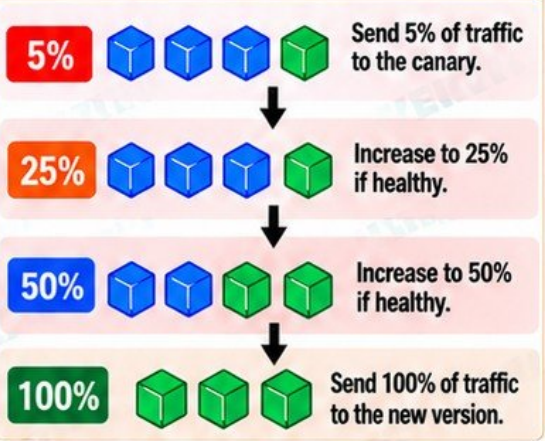


- Receives a small portion of traffic.
- New version being tested in real production.
- Monitored closely for errors and performance.
- Traffic is increased gradually if healthy.

3. How Canary Traffic Routing Works



4. Progressive Canary Rollout



5. Why Start with a Small Percentage?

- Limits the impact if the new version has issues.
- Reduces the number of users affected by bugs or performance problems.
- Gives engineers real production data and user behavior.
- Helps identify issues that were not found in testing.
- A small rollout reduces risk, but it does not eliminate it. Issues can still affect users, data, or external systems.

6. Things to Monitor During Canary

- Error rates and failed requests.
- Application performance (latency, response time).
- CPU, memory, and resource usage.
- Business metrics (sign-ups, purchases, etc.).
- Logs and unusual behavior.
- User feedback and support tickets.

7. Key Benefits

- Real production testing with live traffic.
- Lower risk compared to a full release.
- Quick rollback if issues are detected.
- Better understanding of how the new version behaves with real users.

8. Junior Engineer Insight



Canary deployments help engineers learn how a new version behaves under real traffic before wider exposure. You can see actual user impact, identify issues early, and make more informed decisions about the rollout.

9. Important Considerations

- Requires good monitoring and alerting.
- Some issues may not appear immediately.
- Not all problems can be easily rolled back (e.g. database changes).
- Traffic routing rules must be configured correctly.
- Ensure the canary and stable versions are compatible with shared dependencies.

How Canary Traffic Routing Actually Works

VERIQTA

1. Percentage-Based Routing

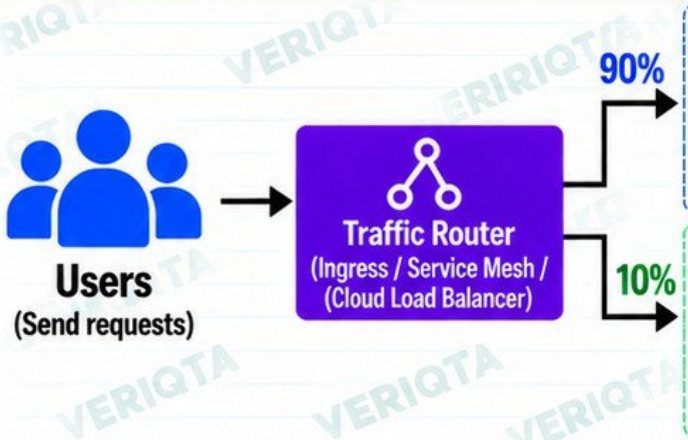
- Traffic is split based on a percentage (e.g. 90% to stable, 10% to canary).
- The router forwards each incoming request to one of the versions using configured rules.
- This happens for every request, not based on the number of Pods.
- The percentage can be changed over time to increase or decrease canary traffic.

2. Traffic Weight vs Number of Replicas

- Traffic weight (percentage) controls how many requests go to each version.
- It is independent of the number of application replicas.
- You can send 10% of traffic to the canary even if it has 1 Pod or many Pods.
- The router decides where to send requests based on weights, not Pod count.

Example: 10% traffic to canary does not mean 1 out of 10 Pods. It means 10% of incoming requests are routed to the canary version.

3. How Canary Traffic Routing Works (Diagram)



- Receives most requests (e.g. 90%).
- Proven and stable.
- Handles normal production traffic.

- Receives a small portion of requests (e.g. 10%).
- New version being tested.
- Monitored for errors, performance, and behavior.

4. Routing Technologies

- **Ingress Controllers** (e.g. NGINX, Traefik): Route traffic based on weights or rules.
- **Service Meshes** (e.g. Istio, Linkerd): Provide advanced traffic splitting, retries, and observability.
- **Cloud Load Balancers** (e.g. AWS ALB, GCP, Azure): Support weighted target groups.
- All of these can be used to send a percentage of traffic to the canary version.

5. Why 10% May Not Be Exactly 10%

- Traffic is distributed per request, not per user.
- In a small sample, you might see 8% or 12% instead of exactly 10%.
- Randomness and load balancing algorithms affect distribution.
- Over time, the percentage gets closer to the configured value.

6. Session Affinity and User Consistency

- Session affinity (sticky sessions) keeps the same user on the same version.
- This is useful when users have a login session, shopping cart, or in-progress work.
- Without session affinity, a user's requests might go to different versions.
- Session affinity can be configured at the ingress, load balancer, or service mesh.

7. Beginner Mistake

Assuming one canary Pod among ten Pods always receives exactly 10% of requests.

- Request distribution is based on traffic weight, not Pod count.
- Actual traffic can be higher or lower than 10% in a short period.
- The router may send multiple requests to the same Pod.

8. Key Takeaways

- Canary routing sends a percentage of requests to the new version.
- Traffic weight is different from the number of replicas.
- Various tools (ingress, service mesh, cloud load balancer) can be used.
- Actual request distribution can vary.
- Session affinity helps keep users consistent.

9. Production Awareness

A 10% traffic allocation does not guarantee exactly 10% of requests in a small sample.

- Monitor real traffic, errors, and user behavior.
- Use session affinity if user consistency is important.
- Understand how your routing tool distributes traffic.
- Adjust traffic gradually based on observed results.

Your First Canary Deployment with Kubernetes

VERIQTA

1. Prepare Stable and Canary Versions

- Create two versions of a sample application with different responses.
- The stable version (v1) serves most users.
- The canary version (v2) is the new version tested with a small percentage of traffic.
- Use version labels to identify each version.

Stable Deployment (v1)

```
kubectl create deployment
myapp-stable --image=nginx:1.0
--replicas=3 --label=app=myapp,
version=stable
```

Canary Deployment (v2)

```
kubectl create deployment
myapp-canary --image=nginx:2.0
--replicas=3 --label=app=myapp,
version=canary
```

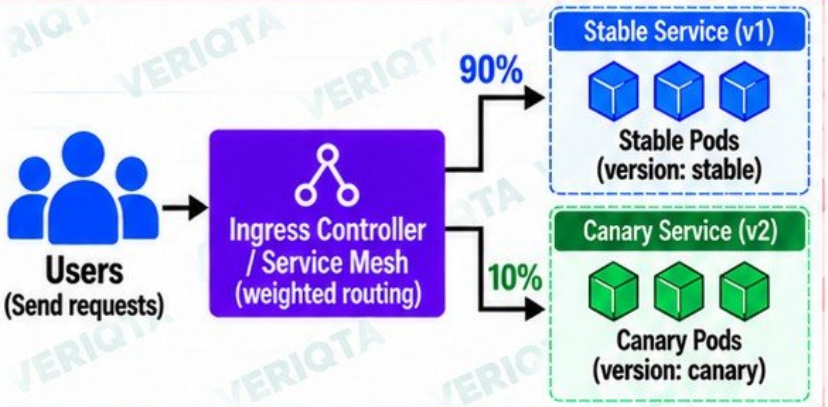
2. Why a Standard Deployment Is Not Enough

- A standard Kubernetes Deployment and Service send traffic to all matching Pods.
- They do not provide precise traffic-weight controls (e.g. 90% to stable, 10% to canary) by themselves.
- To do canary deployments, you need a traffic-routing solution such as an Ingress controller, service mesh, or cloud load balancer that supports weighted routing.

3. Choose a Traffic-Routing Solution

- Use a solution that supports weighted traffic for Kubernetes.
- Examples: NGINX Ingress Controller, Istio, Linkerd, or a cloud load balancer (AWS ALB, GCP, Azure).
- The exact configuration depends on the tool you use.

4. Diagram: Canary Deployment with Weighted Routing



5. Example: NGINX Ingress Weighted Routing

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: myapp-ingress
spec:
  rules:
  - host: myapp.example.com
    http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: myapp-stable
            port:
              number: 80
            # Canary route with 10% weight (NGINX annotation)
annotations:
  nginx.ingress.kubernetes.io/canary: "true"
  nginx.ingress.kubernetes.io/canary-weight: "10"
  nginx.ingress.kubernetes.io/canary-by-header: "canary"
  nginx.ingress.kubernetes.io/canary-by-header-value: "always"
---
# Separate Ingress or annotation for stable with 90%
# (tool-specific configuration may vary)
```

6. Verify Responses from Both Versions

- Send multiple requests and check the responses.
- You should see mostly the stable version and a small number of responses from the canary version.

```
# Send multiple requests
for i in {1..20}; do curl -s http://myapp.example.com; done

# Example responses
Version: stable
Version: stable
Version: canary
Version: stable
...
```

7. Practice Lab

1. Deploy the stable version (v1) with 3 replicas.
2. Deploy the canary version (v2) with 3 replicas.
3. Configure a traffic router (e.g. NGINX Ingress) to send 90% of traffic to stable and 10% to canary.
4. Send multiple requests and observe the responses.
5. Adjust the traffic weight and test again.
6. Keep both deployments running.

8. Production Awareness

Validate the routing controller and its configuration before attempting production traffic changes.

- Test in a non-production environment first.
- Monitor traffic distribution, errors, and performance.
- Ensure the routing rules behave as expected.
- Have a rollback plan if the canary version causes issues.
- Remember that traffic routing does not fix application bugs, security issues, or database compatibility problems.

Key Takeaways

- A standard Kubernetes Deployment does not provide precise traffic-weight controls.
- Use a supported traffic-routing solution.
- Start with a small percentage (e.g. 10%).
- Verify responses from both versions.
- Test and validate before production.

VERIQTA

Canary Analysis: When Should You Continue or Stop?

1. How Engineers Evaluate a Canary

- Engineers use real production data to compare the canary with the stable version.
- They look at errors, performance, and application behavior.
- The goal is to find problems early before increasing traffic.
- Decisions are based on evidence, not assumptions.

2. Key Metrics to Monitor

- **HTTP 5xx errors:** Increase in server errors.
- **Latency:** Response time (average, p95, p99).
- **Request volume:** Make sure the canary receives enough traffic to get meaningful data.
- **Application-specific metrics:** e.g. successful logins, orders, payments, or business conversions.
- **Resource usage:** CPU, memory, and memory limits.

3. Compare Canary vs Stable Version

Stable Version (v1)		Canary Version (v2)	
Requests	95,000	Requests	5,000
Error rate (5xx)	0.2%	Error rate (5xx)	1.2%
Average latency	120 ms	Average latency	250 ms
p95 latency	300 ms	p95 latency	700 ms
Successful logins	99.8%	Successful logins	98.9%

4. Progressive Evaluation Flow

Release 5% Traffic → Send a small percentage (e.g. 5%) to the canary.

Observe Metrics → Monitor errors, latency, and application metrics for a set period.

Evaluate Results → Compare with the stable version and decide if performance is acceptable.

Increase or Stop → If healthy, increase traffic (e.g. 25%, 50%, 100%). If issues are found, stop promotion.

5. Sample Size and Observation Period

- Low traffic can give misleading results.
- A small number of requests may not show real problems or may make problems look worse than they are.
- Observe for an appropriate time based on your application and traffic volume (minutes, hours, or days).
- Consider normal traffic patterns (peak vs off-peak).

6. Example: Monitoring Metrics

```
# Example metrics from monitoring (simplified)
```

Metric	Stable (v1)	Canary (v2)
Requests	95,000	5,000
HTTP 5xx error rate	0.2%	1.2%
Average latency	120 ms	250 ms
p95 latency	300 ms	700 ms
Successful logins	99.8%	98.9%

7. When to Continue or Stop?

Continue Increasing Traffic

- Error rates are similar or lower than the stable version.
- Latency is acceptable and stable.
- Application metrics meet expectations.
- No unusual behavior in logs or monitoring.
- The canary has been running for a sufficient time with enough traffic.

Stop and Investigate

- Higher HTTP 5xx error rate.
- Significant increase in latency.
- Failed transactions or lower success rate.
- Unexpected behavior in logs.
- Errors in specific features or user groups.
- Insufficient data or unclear results (keep observing).

8. Troubleshooting Scenario

The canary reports more errors than the stable version.

1. Check the application logs for errors.
2. Compare recent changes between versions.
3. Verify database connections and external services.
4. Check resource usage (CPU, memory).
5. Look for configuration or dependency issues.
6. Stop promotion until the cause is understood and resolved.

9. Key Takeaways

- Use real metrics and compare with the stable version.
- Give the canary enough traffic and time.
- Watch for both technical and business metrics.
- Stop and investigate if you see higher errors or unacceptable performance.
- Make decisions based on evidence, not assumptions.

10. Production Awareness

- Validate your monitoring and alerting before running a canary.
- Ensure you can roll back or stop traffic quickly if needed.
- Understand your application's normal traffic patterns.
- Canary analysis only works if you monitor the right metrics.
- Do not rely on a single metric or a very short observation period.

VERIQTA

Readiness, Liveness, Startup, and Graceful Shutdown

1. Readiness Probe

- Checks if the application is ready to receive traffic.
- Controls whether the Pod is eligible for Service traffic.
- If the readiness probe fails, the Pod is removed from Service endpoints.
- The Pod keeps running but does not receive new traffic until the probe succeeds.

2. Liveness Probe

- Checks if the application is still running correctly.
- If the liveness probe fails, Kubernetes restarts the Pod.
- Helps recover from application hangs or deadlocks.
- A poorly configured liveness probe can cause frequent restarts (restart loops).

3. Startup Probe

- Checks if the application has started successfully.
- Used for applications that take a long time to start.
- Prevents liveness and readiness probes from running too early.
- Gives the application enough time to initialize before other probes are evaluated.

4. How Probes Work Together

Startup Probe	Readiness Probe	Liveness Probe
Runs first to allow the application time to start.	Determines if the Pod can receive traffic from the Service.	Checks if the application is still healthy and restarts the Pod if it fails.

5. Pod Lifecycle with Probes and Shutdown (Diagram)

Pod Starts	Startup Check	Ready	Receives Traffic	Drains	Stops
Container is created and starts running.	Startup probe verifies the application has started.	Readiness probe succeeds. Pod is added to Service endpoints.	Pod handles application requests.	During shutdown, Pod stops receiving new traffic and finishes in-flight requests.	After the grace period, the container is terminated.

6. Termination Grace Period and Graceful Shutdown

- When a Pod is deleted, Kubernetes sends a SIGTERM signal to the container.
- The Pod is removed from Service endpoints so it stops receiving new traffic.
- The application should handle the signal, finish in-flight requests, and shut down gracefully.
- terminationGracePeriodSeconds defines how long Kubernetes waits before forcing a SIGKILL.
- Use preStop hooks and proper application shutdown logic to release resources and complete ongoing work.

7. Why Healthy Pods Can Still Have Interrupted Connections

- A Pod can be removed from Service endpoints while existing connections are still active.
- Long-running requests may be terminated if the application does not handle shutdown properly.
- Load balancers or proxies may close connections during scaling or updates.
- Network timeouts or client retries can also cause interruptions even when Pods are healthy.

8. Common Failures

- Readiness probe is misconfigured and the Pod receives traffic before the application is ready.
- Liveness probe is too aggressive, causing restart loops.
- Startup probe is not used for slow-starting applications.
- The application exits before completing active requests.
- In-flight requests are lost due to a short termination grace period.

9. Example Probe Configuration (HTTP)

```
readinessProbe:
  httpGet:
    path: /ready
    port: 8080
  initialDelaySeconds: 5
  periodSeconds: 10

livenessProbe:
  httpGet:
    path: /health
    port: 8080
  initialDelaySeconds: 10
  periodSeconds: 10

startupProbe:
  httpGet:
    path: /startup
    port: 8080
  failureThreshold: 30
  periodSeconds: 10
```

10. Key Takeaways

- Use all three probes for better application reliability.
- Readiness controls Service traffic, liveness restarts unhealthy Pods, startup handles slow initialization.
- Configure termination grace periods and shutdown logic.
- Test probe settings in a non-production environment.

11. Production Awareness



- Tune probe timings based on your application behavior.
- Avoid aggressive liveness probes that cause restart loops.
- Ensure graceful shutdown to complete in-flight requests.
- Monitor probe failures, restarts, and terminated requests.
- Healthy Pods can still experience interrupted connections.

VERIQTA

Database Changes: The Hidden Deployment Risk

1. Why Coordinate Application and Database Changes?

- Application and database changes are tightly connected.
- A new application version may depend on new database schema or data.
- Both old and new application versions may run at the same time during a deployment.
- If the database change is not compatible, the old version can break and cause errors.
- Coordinating changes reduces downtime and prevents production incidents.

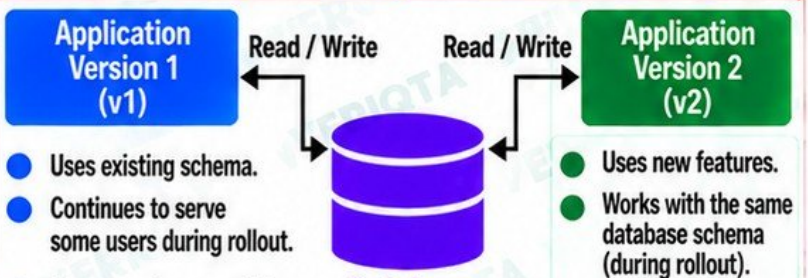
2. The Expand-and-Contract Approach

1 Expand (Backward-Compatible Change)
- Add new columns or tables.
- Keep existing columns and logic.
- Both old and new application versions can work.

2 Migrate (Deploy New Version)
- Deploy the new application version.
- Write to both old and new structures if needed.
- Verify that the new version works correctly.

3 Contract (Cleanup After Rollout)
- Remove old columns or tables.
- Update application code to stop using old data.
- Do this only after the old version is no longer running.

3. Old and New Versions Using the Same Database (Diagram)



- Uses existing schema.
- Continues to serve some users during rollout.
- Must remain compatible with the database.

- Uses new features.
- Works with the same database schema (during rollout).
- Both versions can run together until rollout is complete.

4. Why Removing a Column Too Early Is Risky

Before Removal (Safe)

```
id
name
email
phone  ← Used by v1
created_at
```

After Removal (Breaks Old Version)

```
id
name
email
created_at
```

- If the phone column is removed too early, Application v1 may fail because it still expects this column.
- This can cause errors, crashes, or data loss.
- Always remove old columns only after the old application version is no longer running.

5. Data Migrations, Backups, and Recovery

- Data migrations may be required to move or transform existing data.
- Test migrations in a non-production environment first.
- Always take a backup before making significant changes.
- Understand your recovery options and how to restore data if needed.
- Some database changes (e.g. dropping columns) can be irreversible or require complex recovery steps.

6. Common Database Changes During Deployments

- Adding a new column (backward-compatible).
- Adding a new table.
- Updating data formats or constraints.
- Backfilling data for the new version.
- Removing old columns or tables (after rollout).
- Updating indexes or performance settings.

7. Risks and Common Mistakes

- Making non-compatible schema changes.
- Removing columns too early.
- Running migrations without testing.
- Not taking backups.
- Assuming you can easily roll back database changes.
- Forgetting that both old and new versions may be running at the same time.

8. Key Takeaways

- Plan database changes together with application deployments.
- Use the expand-and-contract approach for compatibility.
- Support both old and new versions during the rollout.
- Always test, back up, and have a recovery plan.

9. Production Awareness

Rolling back application code does not automatically roll back database migrations.

- Database changes may be permanent or difficult to reverse.
- Always validate changes and understand the impact before deploying to production.
- Have a rollback and recovery strategy for database changes.

VERIQTA

Rolling vs Blue-Green vs Canary: Which Should You Choose?

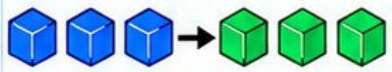
1. Comparison Overview

Feature	Rolling Updates	Blue-Green Deployment	Canary Deployment
How it works	Gradually replaces old version with new version.	Runs two separate environments (Blue and Green) and switches traffic.	Releases new version to a small portion of traffic first.
Infrastructure needs	Single environment.	Two full environments (higher cost).	Single environment (or separate canary deployment).
Traffic control	Automatic, Pod-by-Pod.	Switch all traffic at once (via Service, load balancer, or DNS).	Percentage-based traffic (e.g. 5%, 10%, 25%, 50%, 100%).
Rollback option	Rolls back to previous version.	Switch back to Blue quickly.	Stop and return all traffic to stable version.
Operational complexity	Low.	Medium to high.	Medium.
Best use case	Routine, compatible releases with low risk.	Major changes, environment isolation, and a prepared fallback.	Controlled exposure and real production feedback with lower risk.

2. When to Use Rolling Updates

- The new version is backward-compatible.
- Changes are small and low risk.
- You want a simple and fast deployment process.
- Suitable for many routine updates.
- Kubernetes gradually replaces old Pods with new Pods.

Rolling Update



Old Pods are gradually replaced with new Pods.

3. When to Use Blue-Green Deployment

- You need environment isolation.
- The application has major changes or higher risk.
- You want a fully tested, ready fallback environment.
- You can afford the extra infrastructure cost.
- Traffic is switched all at once.

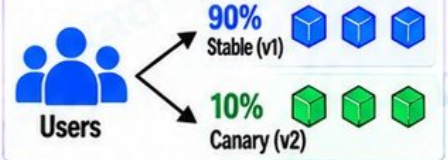
Blue-Green Deployment



4. When to Use Canary Deployment

- You want to release to a small portion of real users first.
- You need to measure performance, errors, and user behavior.
- You want to reduce risk before full rollout.
- It works well when you have good monitoring and observability in place.
- You can gradually increase traffic if metrics look good.

Canary Deployment



5. Decision Scenario

You manage a customer-facing application with strict availability requirements.
A new version includes major changes and uses a new database field, but you want to minimize risk while still testing in production.

Which deployment strategy should you choose?

- **A** Rolling update — Good for small, compatible changes, but higher risk for major changes.
- **B** Blue-green deployment — Provides full environment isolation and a quick rollback option.
- **C** Canary deployment — Releases to a small audience first and allows you to validate with real traffic and metrics.

6. Key Factors to Consider

- Size and risk of the changes.
- Backward compatibility with existing version.
- Infrastructure cost and capacity.
- Need for environment isolation.
- Ability to control and monitor traffic.
- Rollback requirements and recovery time.
- Impact on users if something goes wrong.
- Your team's operational experience.

7. Summary Comparison

Strategy	Main Advantage	Main Disadvantage
Rolling	Simple and low cost	Higher risk for major changes
Blue-Green	Full isolation and quick rollback	Higher infrastructure cost and complexity
Canary	Lower risk with real user feedback	Requires good monitoring and traffic control

8. Junior Engineer Insight

No single deployment strategy is always safest. Choose based on your application architecture, available infrastructure, compatibility, observability, and recovery requirements.

9. Production Awareness

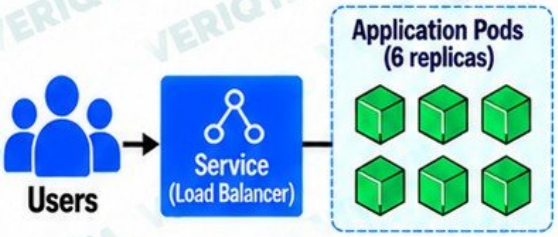
- Understand the impact on users.
- Ensure proper monitoring and alerting.
- Have a clear rollback plan.
- Test in a non-production environment.
- Consider database changes and compatibility.

VERIQTA

Production Deployment Challenge: You Are the On-Call Engineer

1. Scenario

A production application has six healthy replicas.
A new release begins. Within minutes, the error rate increases and some users experience failed requests.

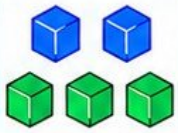


Symptoms:

- Error rate increased.
- Some users see failed kequests.
- Happened within minutes after the new release began.

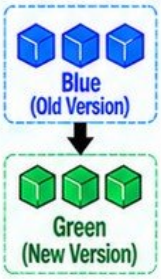
2. Three Deployment Strategies at a Glance

Rolling Update



Replace application instances gradually while keeping enough healthy instances available.

Blue-Green



Prepare two versions separately, then switch production traffic to the new environment.

Canary



Expose a small portion of traffic to the new version, observe its health, then gradually increase exposure.

3. Your Tasks as the On-Call Engineer

1. Which deployment strategy is in use?
2. What evidence should be inspected first?
3. Should the rollout continue, pause, or reverse?
4. Which Kubernetes commands help confirm application and rollout state?
5. How should you verify that service health has recovered?
6. What should be documented before the next deployment?

4. Useful Kubernetes Commands

```
# Check deployment status
kubectl get deployments
kubectl describe deployment <name>
kubectl rollout status deployment/<name>
# Check pods and versions
kubectl get pods -l app=<name> -o wide
kubectl describe pod <pod-name>
# Check Service and endpoints
kubectl get svc <name>
kubectl get endpoints  <name>
# Check recent events
kubectl get events --sort-by=.lastTimestamp
# View application logs
kubectl logs <pod-name> --tail=100
```

5. What to Inspect First?

- Application error rate (e.g. HTTP 5xx).
- Pod status and restarts.
- Pod logs for application errors.
- Deployment rollout status.
- Service endpoints and traffic routing.
- Recent Kubernetes events.
- Latency and response time.
- Infrastructure issues (nodes, network, storage).
- Changes in database or external dependencies.

6. Decision: Continue, Pause, or Reverse?

Continue

- Error rate is normal or within acceptable limits.
- New version is performing as expected.
- No user impact.

Pause

- Error rate is elevated but not critical.
- You need more time to investigate.
- Limits further impact while you gather evidence.

Reverse

- High error rate or user impact.
- Application is unhealthy.
- Serious bugs or failed dependencies.
- New version cannot serve traffic safely.

7. Verify Service Health Has Recovered

- Error rate returns to normal levels.
- Latency is within acceptable range.
- All intended replicas are healthy and ready.
- Service endpoints are correct.
- Users can successfully access the application.
- Business-critical transactions work as expected.
- No new errors in logs or monitoring.

8. Document Before the Next Deployment

- What happened and the timeline.
- Root cause and contributing factors.
- What was changed (application, config, database, infrastructure).
- How it was detected.
- Actions taken (pause, rollback, fix).
- What could have been done earlier.
- Updates to runbooks and monitoring.
- Lessons learned.

9. Release Checklist (Before, During, and After)

- [x] Confirm baseline application health.
- [x] Validate new-version readiness and compatibility.
- [x] Ensure rollback or recovery options are prepared.
- [x] Monitor errors, latency, and business-critical behavior.
- [x] Stop or reverse an unhealthy rollout.
- [x] Verify recovery and record the outcome.

10. Final Engineering Lesson



A deployment is complete only when the intended change is running and the application has been verified to remain healthy.